

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Sistema de Memoria

Universidad Nacional de La Plata



Facultad de Informática





1

Limitaciones del sistema de memoria

2

El principio de localidad

3

La memoria caché

Jerarquía de memoria. Contención. Coherencia: Soluciones hardware al problema de coherencia, False Sharing. Mitigación de contención y coherencia en software.

4

Principio de localidad aplicado

Multiplicación de matrices cuadradas. Matrices triangulares.

A dark, industrial setting, possibly a control room or a laboratory. In the foreground, a desk is cluttered with various items: an open book with dense text, a pen, and some papers. In the background, there are stacks of electronic equipment, glowing lights, and a lamp hanging over the desk. The overall atmosphere is mysterious and technical.

1

Limitaciones del sistema de memoria

Limitaciones del Sistema de memoria

4

- En muchas aplicaciones, la limitación proviene del sistema de memoria.
- Dos parámetros del sistema de memoria:
 - **Latencia:** Tiempo transcurrido desde una solicitud de memoria (*memory request*) hasta que los datos están disponibles para el procesador.
 - **Ancho de banda:** Cantidad de datos por unidad de tiempo que pueden transferirse desde el sistema de memoria al procesador.
- *Memory Wall*: Procesadores evolucionaron mas rápido que el sistema de memoria.
- Problemas si el procesador y la memoria están conectados directamente:
 - Un procesador realiza varias instrucciones por ciclo pero el acceso a memoria por operandos requiere mas de un ciclo (*Tabla libro de Brendan Gregg*).
 - Afecta el rendimiento del procesador porque queda ocioso.

Procesador

Memoria



2

El principio de localidad

El principio de localidad

6

- **Principio de localidad:** Los accesos a memoria de un determinado programa no se distribuyen de manera uniforme por todo el espacio de direcciones, sino que tienden a concentrarse temporalmente en posiciones cercanas.
- El principio de localidad se manifiesta en dos formas:
 - **La localidad temporal:** Tendencia de los programas a reutilizar datos o instrucciones que han sido accedidos recientemente. Un dato consultado hace poco es muy probable que sea utilizado nuevamente en un futuro próximo.
 - **La localidad espacial:** Relacionado con la proximidad de los datos en memoria. Al acceder a un dato, es muy probable que otros datos ubicados en direcciones contiguas se necesiten en un futuro próximo.
- El principio de localidad es la base para entender cómo funcionan y por qué son efectivas las memorias caché.

La memoria caché

Jerarquía de memoria.

Contención.

Coherencia:

Soluciones hardware al problema de coherencia.

False Sharing.

Mitigación de contención y coherencia en software.

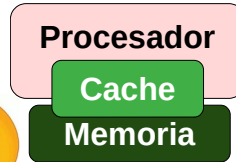


La memoria Cache

8

- **Memoria caché:** memoria entre memoria principal y procesador, que aprovecha el principio de localidad:

- **Ventaja:** Mayor velocidad que la memoria principal.
- **Desventaja:** Tamaño limitado y costo elevado respecto a la memoria principal.



- Ante un requerimiento de memoria, pueden ocurrir dos situaciones:

- **Cache miss (fallo):** El dato no se encuentra en cache, debe traerse desde memoria RAM.



- *La memoria caché esta compuesta por unidades llamadas **líneas de memoria** que pueden contener varios datos. Siempre que se lee una linea de memoria principal se traen datos contiguos.*

- **Cache hit (acierto):** El dato se encuentra en cache, acceso con menor latencia y mejora en el tiempo de respuesta.

- **Objetivo:** idear estrategias que minimicen accesos a memoria principal y maximicen accesos a memoria cache.

La memoria caché

Jerarquía de memoria.

Contención.

Coherencia:

Soluciones hardware al problema de coherencia.

False Sharing.

Mitigación de contención y coherencia en software.

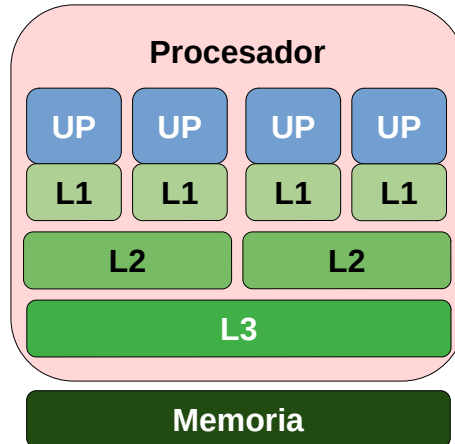


La memoria Cache

Jerarquía de memoria

10

- Primeras arquitecturas: *Caché off-chip*. *Chip* externo en el *motherboard*.
- A partir de 1989: *Caché on-chip*. Dentro del circuito integrado del procesador.
- En procesadores modernos, con varias unidades de procesamiento (cores), es habitual encontrar más de un nivel de cache (L1, L2, L3) conformando lo que se conoce como **jerarquía de memoria** (RAM + Caches).
 - A mayor nivel, mayor latencia y mayor tamaño.



La memoria caché

Jerarquía de memoria.

Contención.

Coherencia:

Soluciones hardware al problema de coherencia.

False Sharing.

Mitigación de contención y coherencia en software.



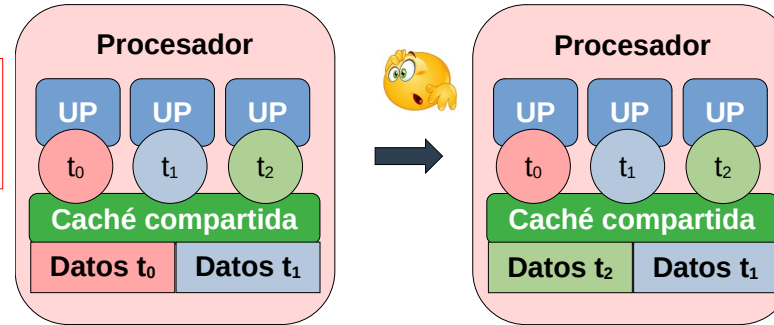
La memoria Cache

Contención

12

- **Contención:** Conflicto en acceso a un recurso compartido (Memoria, disco, buses, red)
- Contención al usar **cachés compartidas**: Aplicación paralela con varios hilos compartiendo caché.

El **Hilo t_2** intenta traer sus datos de memoria pero la caché está llena.



Se activa el algoritmo de reemplazo que, por ejemplo, invalida los datos del **Hilo t_0** .

Si esto se repite se degrada el rendimiento (**caché trashing**)

- Los sistemas operativos resuelven problemas de contención pero no todos.
- Alternativas:
 - Crear menos hilos.
 - Distribuyendo los hilos de manera de aprovechar mejor la localidad.
 - Utilizando software para particionado de memoria cache.
 - Rediseñando el algoritmo para obtener patrones de acceso libres de contención.

La memoria caché

Jerarquía de memoria.

Contención.

Coherencia:

Soluciones hardware al problema de coherencia.

False Sharing.

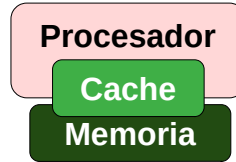
Mitigación de contención y coherencia en software.



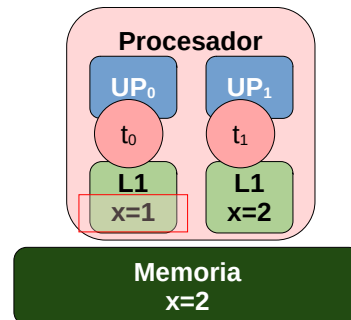
La memoria Cache Coherencia

14

- Una lectura de memoria debe retornar el último valor escrito en esa dirección, sin importar el hilo o proceso que escribió dicho valor:
 - En sistemas de una unidad de procesamiento, no hay interferencias. Hilos o procesos ejecutan de a uno a la vez y ven la misma jerarquía de memoria
 - En sistemas con varias unidades de procesamiento, hilos o procesos deberían ver el mismo estado de la memoria independientemente de si estos se ejecutan o no en diferentes unidades de procesamiento.



Problema de coherencia: Valores en caché desactualizados respecto a los valores en memoria.



La memoria Cache

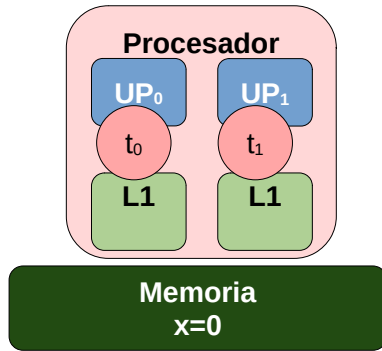
Coherencia - Ejemplo

15

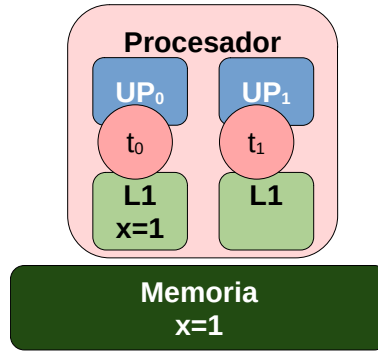
- **Ejemplo:** varios hilos actualizan un variable atómicamente:

```
x = 0
```

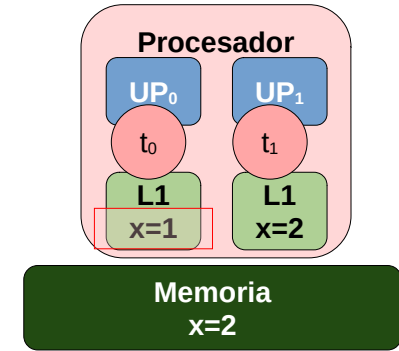
```
Hilo t0 {  
  ...  
  atomic(x=x+1)  
  ...  
}  
  
Hilo t1 {  
  ...  
  atomic(x=x+1)  
  ...  
}
```



Inicialmente, una variable reside en memoria inicializada en 0.



Un hilo (t₀) lee e incrementa la variable. Se actualiza el valor en memoria principal y se almacena en la cache privada del hilo.



Otro hilo (t₁) lee e incrementa la variable. Se actualiza el valor en memoria principal y se almacena en la cache privada del hilo, pero queda desactualizado en las caches de otros hilos.

El problema de coherencia es un problema de **hardware** independiente de la sincronización del programa.

La memoria caché

Jerarquía de memoria.

Contención.

Coherencia:

Soluciones hardware al problema de coherencia.

False Sharing.

Mitigación de contención y coherencia en software.



- Problemas de coherencia se resuelven con protocolos implementados en *hardware*.
- Dos tipos de protocolos de coherencia:
 - **Sistemas basados en bus (snoopy):** Continuamente se observa por cambios en las líneas de cache.
 - **Sistemas basados en diccionario:** Un diccionario almacena la validez de las líneas de cache.
- Ante un problema de coherencia, los sistemas siguen alguna de las siguientes políticas:
 - **Basadas en invalidación (MSI, MESI, MOSI, MOESI):** Un cambio en el valor de una variable almacenada en cache de una unidad de procesamiento, genera la **invalidación** de las líneas de cache de toda unidad de procesamiento que tengan una copia de esa variable.
 - **Basadas en actualización (Dragon):** Un cambio en el valor de una variable almacenada en cache de una unidad de procesamiento, genera la **actualización** de las líneas de cache de toda unidad de procesamiento que tengan una copia de esa variable.

La memoria caché

Jerarquía de memoria.

Contención.

Coherencia:

Soluciones hardware al problema de coherencia.

False Sharing.

Mitigación de contención y coherencia en software.



- Formas de mitigar el problema de **False Sharing**:
 - **Alineación de datos:** Asegurarse que las variables accedidas por hilos o procesos diferentes estén en diferentes líneas de caché.
 - **Padding (relleno):** Agregar espacio entre las variables para que ocupen diferentes líneas de caché.
 - **Estrategias de acceso a datos:** Reorganizar el acceso a los datos para que los hilos o procesos operen sobre partes diferentes de la memoria.
 - **Uso de primitivas de sincronización eficientes:** Las primitivas de sincronización (locks, semáforos, barreras, etc.) pueden provocar *false sharing*. Rediseñar el código para que no causen un acceso innecesario a las mismas líneas de caché.
 - **Uso de memoria NUMA (Non-Uniform Memory Access):** En arquitectura NUMA, asegurarse que los hilos o procesos accedan a su memoria local y así evitar que hilos o procesos de diferentes nodos accedan a la misma caché.
 - **Duplicación de estructuras y reducción de frecuencia de escrituras:** Que los hilos o procesos trabajen sobre copias privadas antes de escribirlos.

La memoria caché

Jerarquía de memoria.

Contención.

Coherencia:

Soluciones hardware al problema de coherencia.

False Sharing.

Mitigación de contención y coherencia en software.



La memoria Cache

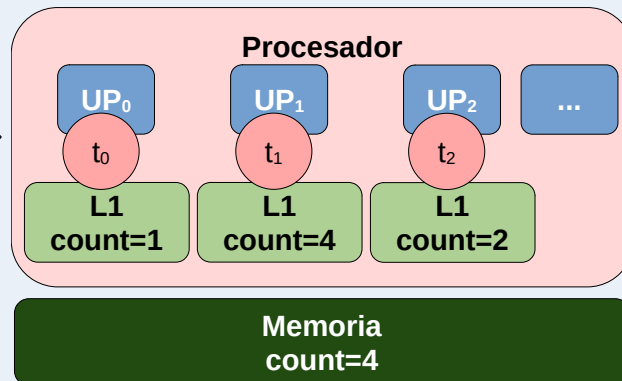
Mitigación de contención y coherencia en software

- **Problema (Práctica 1 de Concurrency y Paralelismo):** Dado un número N calcular cuántas veces aparece ese número en un arreglo v de longitud M . Paralelizar con H hilos sobre H unidades de procesamiento. (M proporcional a H)

```
count = 0  
Array v[M]
```

```
Hilo t[id:0 to H-1]{  
  Part = M/H  
  Ini = id*Part  
  End = Ini + Part - 1  
  
  for i = Ini to End  
    if (v[i]==N)  
      atomic(count=count+1)  
}
```

NO!!!



Peor caso:

Toda posición del arreglo es N .
 M accesos de escritura a la misma posición de memoria.
Se secuencializa (contención alta)
Protocolo de coherencia 100% activo.

```
Hilo t[id:0 to H-1]{  
  local = 0  
  Part = M/H  
  Ini = id*Part  
  End = Ini + Part - 1  
  
  for i = Ini to End  
    if (v[i]==N)  
      local=local+1  
      atomic(count=count+local)  
}
```



Solución: Contar en una variable local minimizando accesos a la variable compartida (contención baja).

- **Empíricamente:**

- Implementamos el algoritmo secuencial y paralelo en pthreads.
- Ejecutamos el peor caso para una carga de trabajo de 2^{30} elementos enteros.
- Pthreads ejecutado con 16 hilos en 16 unidades de procesamiento.
- Tiempos de ejecución medidos en segundos.

Secuencial	Paralelo Variable compartida	Paralelo Variable local
3,41s	140,35s	0,71s



Peor que secuencial!!!

Cuando el objetivo es mejorar el rendimiento, no debemos abstraernos completamente del hardware.





4

Principio de localidad aplicado

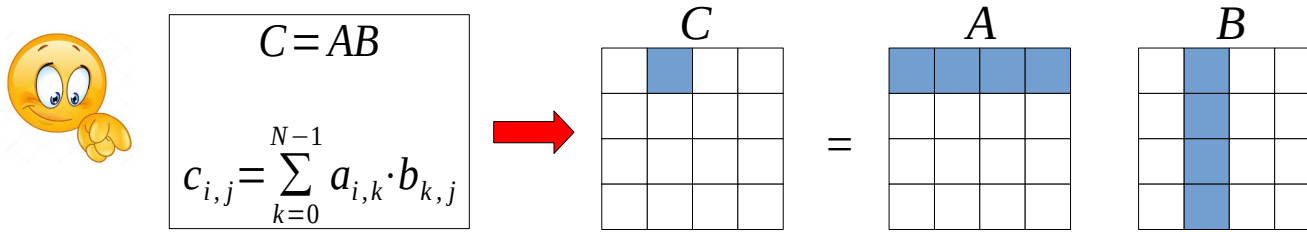
Multiplicación de matrices cuadradas

Matrices triangulares

Principio de localidad aplicado

Multiplicación de matrices cuadradas

- Analizaremos el comportamiento de la memoria cache cuando resolvemos el problema de la multiplicación de dos matrices cuadradas de $N \times N$:



- Por la naturaleza del patrón de acceso y para aprovechar la localidad, almacenamos en memoria por filas (*row-major*) las matrices A y C, y por columnas (*column-major*) la matriz B.

Pseudocódigo

```
for i = 0 to N-1
  for j = 0 to N-1
    C[i,j] = 0
    for k = 0 to N-1
      C[i,j] = C[i,j] + A[i,k]*B[k,j]
```



```
for i = 0 to N-1
  for j = 0 to N-1
    for k = 0 to N-1
      C[i*N+j] = C[i*N+j] + A[i*N+k]*B[k+j*N]
```



¿Por qué?

Principio de localidad aplicado

Multiplicación de matrices cuadradas

26

- Todos los lenguajes de programación permiten declarar matrices.



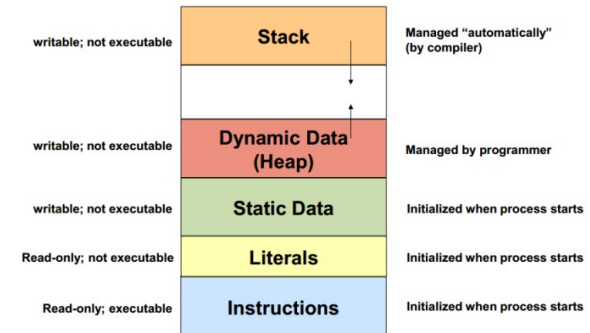
¿Cómo almacenan los lenguajes una matriz de $N \times N$ en memoria?
¿Cómo puede el programador almacenar una matriz por filas o por columnas?

- Declaración de matrices en el lenguaje C:

```
int A [N] [N] ;  
int B [N] [N] ;  
int C [N] [N] ;
```

- Desventajas:

- El lenguaje determina como se almacenan (El lenguaje C siempre por filas).
- Se almacenan en memoria estática (Pila o *Stack*).
- Tamaño de matrices limitado por la Pila.
 - Posibilidad de *stack overflow*.



Principio de localidad aplicado

Multiplicación de matrices cuadradas



- Mayor control definiéndolas:

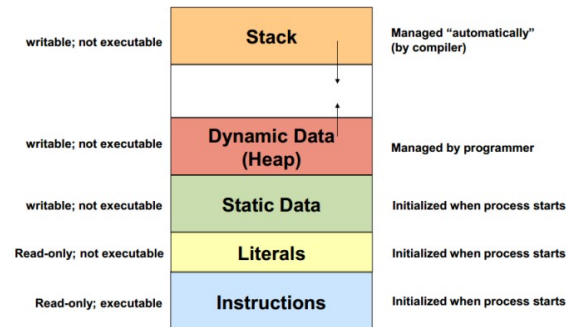
```
tipo* A;
tipo* B;
tipo* C;

...
A= (tipo*)malloc(sizeof(tipo)*N*N);
B= (tipo*)malloc(sizeof(tipo)*N*N);
C= (tipo*)malloc(sizeof(tipo)*N*N);

...
free(A);
free(B);
free(C);
```

- **Ventajas:**

- El programador determina como se almacenan en memoria.
- Se almacenan en memoria dinámica (*Heap*).
- Tamaño de matrices eventualmente “ilimitado”.



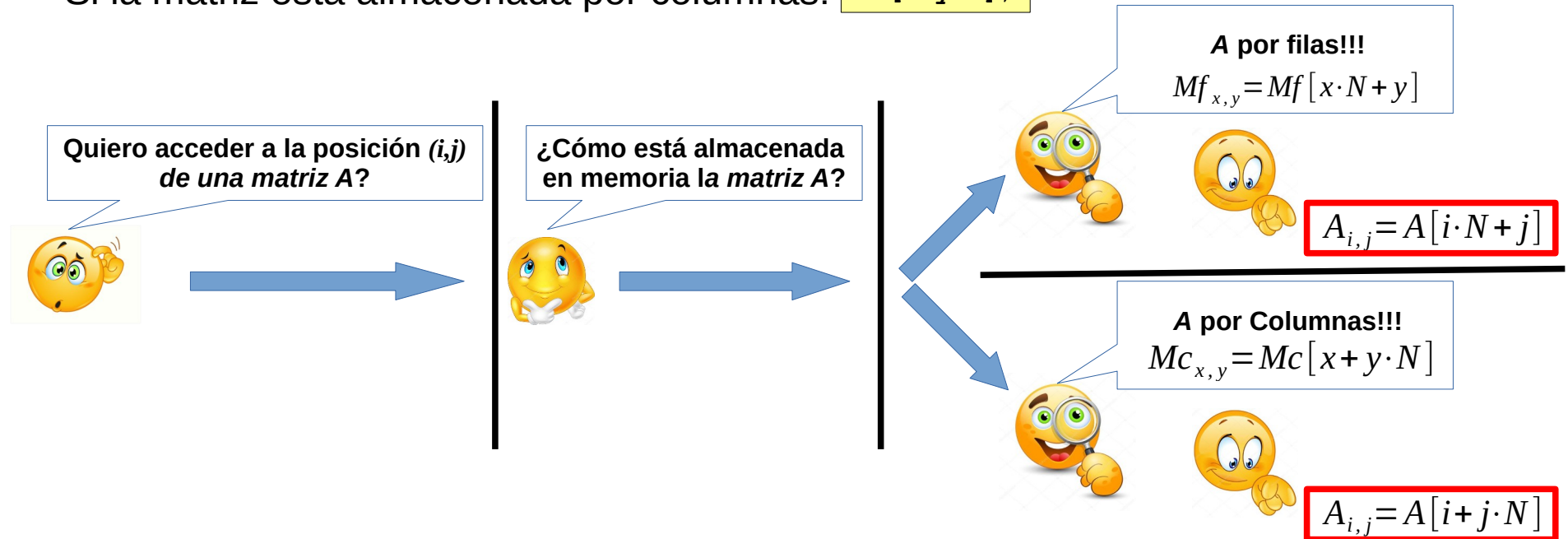
Principio de localidad aplicado

Multiplicación de matrices cuadradas

28

¿Cómo acceder al elemento (x,y) de una matriz M ?

- Si la matriz está almacenada por filas: $Mf[x \cdot N + y]$;
- Si la matriz está almacenada por columnas: $Mc[x + y \cdot N]$;



Principio de localidad aplicado

Multiplicación de matrices cuadradas

29

- Ejemplo: Comparación de dos algoritmos secuenciales, ambos $O(n^3)$:
 - Matrices A, **B** y C almacenadas en memoria por **filas**.
 - La matriz **B** almacenada en memoria por **columnas**.
- Características de la ejecución:
 - Tamaños de matrices N= 512, 1024, 2048
 - Tiempos de ejecución medidos en segundos
 - Medimos fallos de cache de nivel 1:
 - *PAPI (Performance Application Programming Interface)*

N	B por filas		B por columnas	
	Tiempo	Fallos L1	Tiempo	Fallos L1
512	1,60s	135.411.031	1,08s	16.872.076
1024	26,26s	1.097.641.458	8,87s	135.070.148
2048	265,59s	14.005.385.456	71,28s	1.157.777.331



Mas fallos de caché peor tiempo de ejecución

Principio de localidad aplicado

Multiplicación de matrices cuadradas

30



¿Y si multiplicamos $C = AA$?

- Dos opciones (ambas $O(n^3)$):
 - Multiplicamos AA tal como está, A almacenada en memoria por **filas**.
 - Creamos una matriz auxiliar A_{col} almacenada en memoria por **columnas**:
 - Mas consumo de memoria
 - Mas operaciones/instrucciones ¿Mas tiempo de ejecución? (*Libro de Brendan Gregg*)

N	C=AA	C=AA _{col}
512	Tiempo:1,76s Fallos L1:134.637.258 Instrucciones:6.176.380.648	Tiempo:1,25s Fallos L1:17.172.672 Instrucciones:6.182.676.494
1024	Tiempo: 19,91s Fallos L1:1.078.625.820 Instrucciones:49.401.576.181	Tiempo: 10,09s Fallos L1:136.042.686 Instrucciones:49.426.748.663
2048	Tiempo:293,36s Fallos L1:11.588.145.094 Instrucciones:395.174.834.418	Tiempo:89,59s Fallos L1:1.178.020.959 Instrucciones:395.275.469.418

Medimos fallos e instrucciones con PAPI

N=2048, A_{col} ejecuta **100.635.000 de instrucciones adicionales**, **10.410.124.135 menos de fallos de cache.**



Mas instrucciones, Menos fallos
Mejor rendimiento!!!



4

Principio de localidad aplicado

Multiplicación de matrices cuadradas

Matrices triangulares

Principio de localidad aplicado

Matrices triangulares

32

- **Matriz triangular:** Matriz cuadrada donde los elementos por encima o por debajo de la diagonal son cero:

$$U = \begin{pmatrix} u_{0,0} & u_{0,1} & u_{0,2} \\ 0 & u_{1,1} & u_{1,2} \\ 0 & 0 & u_{2,2} \end{pmatrix}$$

Matriz Triangular Superior

$$L = \begin{pmatrix} l_{0,0} & 0 & 0 \\ l_{1,0} & l_{1,1} & 0 \\ l_{2,0} & l_{2,1} & l_{2,2} \end{pmatrix}$$

Matriz Triangular Inferior

- **No almacenar ceros:** requerimientos de memoria críticos y matrices muy grandes.
- Objetivo: minimizar el espacio de almacenamiento

Suponemos que **NUNCA accedemos a las posiciones nulas.**

Cualquier operación que las involucre da como resultado cero (Ejemplo: Multiplicación de matrices).

$$U = \begin{pmatrix} u_{0,0} & u_{0,1} & u_{0,2} \\ 0 & u_{1,1} & u_{1,2} \\ 0 & 0 & u_{2,2} \end{pmatrix} = \begin{pmatrix} u_{0,0} & u_{0,1} & u_{0,2} \\ \textcolor{red}{u_{1,0}}=0 & u_{1,1} & u_{1,2} \\ \textcolor{red}{u_{2,0}}=0 & \textcolor{red}{u_{2,1}}=0 & u_{2,2} \end{pmatrix}$$

Principio de localidad aplicado

Matrices triangulares

33

¿Cómo accedemos a la posición $T[i,j]$ de una matriz triangular almacenada en forma reducida?



- Depende de:
 - Si es triangular superior (U) o inferior (L)
 - Si está almacenada por filas o columnas.
- La combinación de estas características nos da 4 posibilidades:

	Almacenada por filas		Almacenada por columnas																																					
Triangular Superior	<table><tr><td>$u_{0,0}$</td><td>$u_{0,1}$</td><td>$u_{0,2}$</td><td>$u_{1,1}$</td><td>$u_{1,2}$</td><td>$u_{2,2}$</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td colspan="6">Memoria</td></tr></table>	$u_{0,0}$	$u_{0,1}$	$u_{0,2}$	$u_{1,1}$	$u_{1,2}$	$u_{2,2}$	0	1	2	3	4	5	Memoria							<table><tr><td>$u_{0,0}$</td><td>$u_{0,1}$</td><td>$u_{1,1}$</td><td>$u_{0,2}$</td><td>$u_{1,2}$</td><td>$u_{2,2}$</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td colspan="6">Memoria</td></tr></table>	$u_{0,0}$	$u_{0,1}$	$u_{1,1}$	$u_{0,2}$	$u_{1,2}$	$u_{2,2}$	0	1	2	3	4	5	Memoria						
$u_{0,0}$	$u_{0,1}$	$u_{0,2}$	$u_{1,1}$	$u_{1,2}$	$u_{2,2}$																																			
0	1	2	3	4	5																																			
Memoria																																								
$u_{0,0}$	$u_{0,1}$	$u_{1,1}$	$u_{0,2}$	$u_{1,2}$	$u_{2,2}$																																			
0	1	2	3	4	5																																			
Memoria																																								
Triangular inferior	<table><tr><td>$l_{0,0}$</td><td>$l_{1,0}$</td><td>$l_{1,1}$</td><td>$l_{2,0}$</td><td>$l_{2,1}$</td><td>$l_{2,2}$</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td colspan="6">Memoria</td></tr></table>	$l_{0,0}$	$l_{1,0}$	$l_{1,1}$	$l_{2,0}$	$l_{2,1}$	$l_{2,2}$	0	1	2	3	4	5	Memoria							<table><tr><td>$l_{0,0}$</td><td>$l_{1,0}$</td><td>$l_{2,0}$</td><td>$l_{1,1}$</td><td>$l_{2,1}$</td><td>$l_{2,2}$</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr><tr><td colspan="6">Memoria</td></tr></table>	$l_{0,0}$	$l_{1,0}$	$l_{2,0}$	$l_{1,1}$	$l_{2,1}$	$l_{2,2}$	0	1	2	3	4	5	Memoria						
$l_{0,0}$	$l_{1,0}$	$l_{1,1}$	$l_{2,0}$	$l_{2,1}$	$l_{2,2}$																																			
0	1	2	3	4	5																																			
Memoria																																								
$l_{0,0}$	$l_{1,0}$	$l_{2,0}$	$l_{1,1}$	$l_{2,1}$	$l_{2,2}$																																			
0	1	2	3	4	5																																			
Memoria																																								



Principio de localidad aplicado

Matrices triangulares – Superior por filas

34

- **Matriz triangular superior** almacenada **por filas** como un arreglo en memoria.
- El acceso a una nueva fila requiere restar del cálculo $U[i \cdot N + j]$ los elementos en cero hasta esa posición.
- Para una matriz de $N \times N$:

$$U = \begin{pmatrix} u_{0,0} & u_{0,1} & u_{0,2} & \dots & u_{0,n-1} \\ 0 & u_{1,1} & u_{1,2} & \dots & u_{1,n-1} \\ 0 & 0 & u_{2,2} & \dots & u_{2,n-1} \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & u_{n-1,n-1} \end{pmatrix}$$

Ceros hasta la **fila 0: 0**
 Ceros hasta la **fila 1: 1**
 Ceros hasta la **fila 2: 3**
 Ceros hasta la **fila 3: 6**
 Ceros hasta la **fila i-ésima:**

$$\sum_{fila=0}^i fila = \frac{i \cdot (i+1)}{2}$$

Acceder al elemento $U[i,j]$:

$$U[i, j] = U_{arreglo} \left[i \cdot N + j - \frac{i \cdot (i+1)}{2} \right]$$

Ejemplo $N=3$

$$U = \begin{pmatrix} u_{0,0} & u_{0,1} & u_{0,2} \\ 0 & u_{1,1} & u_{1,2} \\ 0 & 0 & u_{2,2} \end{pmatrix}$$

$u_{0,0}$	$u_{0,1}$	$u_{0,2}$	$u_{1,1}$	$u_{1,2}$	$u_{2,2}$
0	1	2	3	4	5

Memoria

	i	j	$U[i, j] = U_{arreglo} \left[i \cdot N + j - \frac{i \cdot (i+1)}{2} \right]$
$u_{0,0}$	0	0	0
$u_{0,1}$	0	1	1
$u_{0,2}$	0	2	2
$u_{1,1}$	1	1	3
$u_{1,2}$	1	2	4
$u_{2,2}$	2	2	5

Principio de localidad aplicado

Matrices triangulares – Inferior por columnas

- **Matriz triangular inferior** almacenada **por columnas** como un arreglo en memoria.
- El acceso a una nueva columna, requiere restar del cálculo $L[i + j \cdot N]$ los elementos en cero hasta esa posición.
- Para una matriz de $N \times N$:



$$L = \begin{pmatrix} l_{0,0} & 0 & 0 & \dots & 0 \\ l_{1,0} & l_{1,1} & 0 & \dots & 0 \\ l_{2,0} & l_{2,1} & l_{2,2} & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ l_{n-1,0} & l_{n-1,1} & l_{n-1,2} & \dots & l_{n-1,n-1} \end{pmatrix}$$

Ceros hasta la **columna 0: 0**
 Ceros hasta la **columna 1: 1**
 Ceros hasta la **columna 2: 3**
 Ceros hasta la **columna 3: 6**
 Ceros hasta la **columna j-ésima:**

$$\sum_{columna=0}^j columna = \frac{j \cdot (j+1)}{2}$$



Acceder al elemento $L[i,j]$:

$$L[i, j] = L_{arreglo} \left[i + j \cdot N - \frac{j \cdot (j+1)}{2} \right]$$

Ejemplo $N=3$

$$L = \begin{pmatrix} l_{0,0} & 0 & 0 \\ l_{1,0} & l_{1,1} & 0 \\ l_{2,0} & l_{2,1} & l_{2,2} \end{pmatrix}$$

$l_{0,0}$	$l_{1,0}$	$l_{2,0}$	$l_{1,1}$	$l_{2,1}$	$l_{2,2}$
0	1	2	3	4	5
Memoria					

	i	j	$L[i, j] = L_{arreglo} \left[i + j \cdot N - \frac{j \cdot (j+1)}{2} \right]$
$l_{0,0}$	0	0	0
$l_{1,0}$	1	0	1
$l_{2,0}$	2	0	2
$l_{1,1}$	1	1	3
$l_{2,1}$	2	1	4
$l_{2,2}$	2	2	5



Principio de localidad aplicado

Matrices triangulares – Inferior por filas

36

- **Matriz triangular inferior** almacenada **por filas** como un arreglo en memoria.
- El acceso a una nueva fila, requiere restar del cálculo $L[i \cdot N + j]$ los elementos en cero hasta esa posición.
- Para una matriz de $N \times N$:



$$L = \begin{pmatrix} l_{0,0} & 0 & 0 & \dots & 0 \\ l_{1,0} & l_{1,1} & 0 & \dots & 0 \\ l_{2,0} & l_{2,1} & l_{2,2} & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ l_{n-1,0} & l_{n-1,1} & l_{n-1,2} & \dots & l_{n-1,n-1} \end{pmatrix}$$

Ceros hasta la **fila 0: 0**
 Ceros hasta la **fila 1: N-1**
 Ceros hasta la **fila 2: 2N-3**
 Ceros hasta la **fila 3: 3N-6**
 Ceros hasta la **fila i-ésima:**

$$i \cdot N - \sum_{fila=0}^i fila = i \cdot N - \frac{i \cdot (i+1)}{2}$$



Acceder al elemento $L[i,j]$:

$$L[i, j] = L_{arreglo} \left[i \cdot N + j - \left(i \cdot N - \frac{i \cdot (i+1)}{2} \right) \right]$$

Ejemplo $N=3$

$$L = \begin{pmatrix} l_{0,0} & 0 & 0 \\ l_{1,0} & l_{1,1} & 0 \\ l_{2,0} & l_{2,1} & l_{2,2} \end{pmatrix}$$

$l_{0,0}$	$l_{1,0}$	$l_{1,1}$	$l_{2,0}$	$l_{2,1}$	$l_{2,2}$
0	1	2	3	4	5
Memoria					

	i	j	$L[i, j] = L_{arreglo} \left[i \cdot N + j - \left(i \cdot N - \frac{i \cdot (i+1)}{2} \right) \right]$
$l_{0,0}$	0	0	0
$l_{1,0}$	1	0	1
$l_{1,1}$	1	1	2
$l_{2,0}$	2	0	3
$l_{2,1}$	2	1	4
$l_{2,2}$	2	2	5

Principio de localidad aplicado

Matrices triangulares – Superior por columnas

37

- **Matriz triangular superior** almacenada **por columnas** como un arreglo en memoria.
- El acceso a una nueva columna, requiere restar del cálculo $U[i + j \cdot N]$ los elementos en cero hasta esa posición.
- Para una matriz de $N \times N$:



$$U = \begin{pmatrix} u_{0,0} & u_{0,1} & u_{0,2} & \dots & u_{0,n-1} \\ 0 & u_{1,1} & u_{1,2} & \dots & u_{1,n-1} \\ 0 & 0 & u_{2,2} & \dots & u_{2,n-1} \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & u_{n-1,n-1} \end{pmatrix}$$

Ceros hasta la **columna 0: 0**

Ceros hasta la **columna 1: N-1**

Ceros hasta la **columna 2: 2N-3**

Ceros hasta la **columna 3: 3N-6**

Ceros hasta la **columna j-ésima:**

$$j \cdot N - \sum_{\text{columna}=0}^j \text{columna} = j \cdot N - \frac{j \cdot (j+1)}{2}$$



Acceder al elemento $U[i,j]$:

$$U[i, j] = U_{\text{arreglo}} \left[i + j \cdot N - \left(j \cdot N - \frac{j \cdot (j+1)}{2} \right) \right]$$

Ejemplo $N=3$



$$U = \begin{pmatrix} u_{0,0} & u_{0,1} & u_{0,2} \\ 0 & u_{1,1} & u_{1,2} \\ 0 & 0 & u_{2,2} \end{pmatrix}$$

$u_{0,0}$	$u_{0,1}$	$u_{1,1}$	$u_{0,2}$	$u_{1,2}$	$u_{2,2}$
0	1	2	3	4	5

Memoria

	i	j	$U[i, j] = U_{\text{arreglo}} \left[i + j \cdot N - \left(j \cdot N - \frac{j \cdot (j+1)}{2} \right) \right]$
$u_{0,0}$	0	0	0
$u_{0,1}$	0	1	1
$u_{1,1}$	1	1	2
$u_{0,2}$	0	2	3
$u_{1,2}$	1	2	4
$u_{2,2}$	2	2	5

Principio de localidad aplicado

Matrices triangulares - Resumen

38

$$U[i, j] = U_{\text{filas}} \left[i \cdot N + j - \frac{i \cdot (i+1)}{2} \right]$$

$$L[i, j] = L_{\text{columnas}} \left[i + j \cdot N - \frac{j \cdot (j+1)}{2} \right]$$

$$L[i, j] = L_{\text{filas}} \left[i \cdot N + j - \left(i \cdot N - \frac{i \cdot (i+1)}{2} \right) \right] = L_{\text{filas}} \left[\cancel{i \cdot N} + j - \cancel{i \cdot N} + \frac{i \cdot (i+1)}{2} \right] = L_{\text{filas}} \left[j + \frac{i \cdot (i+1)}{2} \right]$$

$$U[i, j] = U_{\text{columnas}} \left[i + j \cdot N - \left(j \cdot N - \frac{j \cdot (j+1)}{2} \right) \right] = U_{\text{columnas}} \left[i + \cancel{j \cdot N} - \cancel{j \cdot N} + \frac{j \cdot (j+1)}{2} \right] = U_{\text{columnas}} \left[i + \frac{j \cdot (j+1)}{2} \right]$$

Principio de localidad aplicado

Matrices triangulares

39

¿Cómo multiplicar con matrices triangulares?

- Depende del tipo de multiplicación (LM , UM , ML , MU), debemos definir que posiciones **NO recorrer** por ser **CERO**.
- Suponemos que i y j representan los índices que nos ubican en la fila y columna, respectivamente.
- Las iteraciones **FOR** de i y j , no se ven afectadas porque definen los índices a calcular en la matriz *Resultado* (todas las posiciones!!!).
- Los límites lo define la iteración **FOR** de k .

```
For i = 0 to N-1
  For j = 0 to N-1
    For k = 0 to N-1
      if("k está en los límites")
        "Multiplicar"
```



NO!!!

```
For i = 0 to N-1
  For j = 0 to N-1
    For k = ?? to ??
      "Multiplicar"
```



Sólo recorreremos las posiciones
que no son cero.
Determinarlas en la práctica!!!

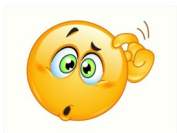
Principio de localidad aplicado

Matrices triangulares - Trade-off (Cómputo - Memoria)

40



- **Trade-off:** situación en la cual aceptamos perder en cierta cualidad a cambio de ganar en otra.
- Cualquier mejora pueden introducir *overhead*. Si el *overhead* no puede evitarse debe minimizarse, para mitigar el impacto negativo en el rendimiento.
- Caso matrices triangulares:
 - **Mejora:** Ganamos espacio de almacenamiento
 - **Overhead:** mas cálculos, mas instrucciones



¿Qué ocurre empíricamente?

Principio de localidad aplicado

Matrices triangulares - Trade-off (Cómputo - Memoria)

41

- **Empíricamente:** Tiempos en segundos de todas las soluciones.

	LM		ML		MU		UM	
N	0	NO 0	0	NO 0	0	NO 0	0	NO 0
512	0,088s	0,085s	0,086s	0,084s	0,087s	0,084s	0,086s	0,084s
1024	0,70s	0,70s	0,69s	0,65s	0,70s	0,65s	0,73s	0,73s
2048	5,88s	5,88s	6,09s	5,83s	5,99s	5,83s	5,99s	5,98s



Mas instrucciones pero...
Menos datos, mejor localidad, menos fallos de caché
Mejor rendimiento!!!

Próxima clase

Capítulo 3

Clasificación de arquitecturas paralelas.

Capítulo 4

Arquitecturas paralelas. Evolución y tendencias.



